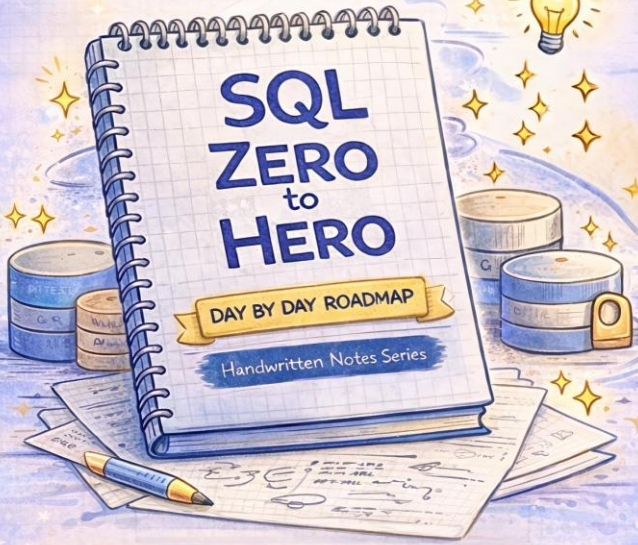
LEARN SQL SERVER

in 14 DAYS



ZERO TO HERO GUIDE

MASTER SQL IN 2 WEEKS!

Pankaj Ikhar
Solution Architect & Mentor

#SQL #Database #DotNet

SQL Zero to Hero – Day 1

SELECT + WHERE Clause

Start your SQL journey with two essential commands to query data:

✔ SELECT

→ Retrieve data **FROM** a table

✔ WHERE

→ Filter rows based on a condition.

Examples:

- SELECT * FROM Employees;

SELECT * FROM Employees;

- SELECT * FROM Employees,
  WHERE Department = 'IT';

Pankaj Ikhar
Solution Architect & Mentor

SQL Zero to Hero – Day 2

LIKE / IN / BETWEEN / AND OR NOT

- ✔ Filter rows using **LIKE** (for patterns), IN, BETWEEN, AND, OR, NOT.
- ✔ Quickly find specific text, multiple values, or a range of values.
- ✔ Combine conditions with logical operators.

Examples:

- SELECT * FROM Customers
  WHERE Name LIKE 'A%"
  → AND City IN ('Pune', 'Mumbai');
- SELECT * FROM Orders
  WHERE Amount BETWEEN 1000 AND 5000;

Pankaj Ikhar
Solution Architect & Mentor

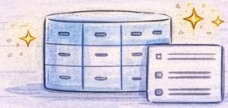
SQL Zero to Hero – Day 3

Aggregate Functions

Quickly analyze data with built-in aggregate functions.

✔ COUNT

→ Counts the number of rows



✔ SUM

→ Calculates the total of a column

✔ AVG

→ Calculates the average value

✔ MIN & MAX

→ Finds the smallest or largest value

Examples:

- SELECT * COUNT(*) FROM Sales;
- SELECT AVG(Amount) FROM Orders;

Pro Tip:

✔ Mix them with GROUP BY for even deeper insights!

Pankaj Ikhar
Solution Architect & Mentor

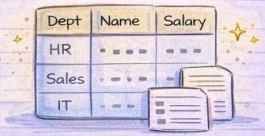
SQL Zero to Hero – Day 4

GROUP BY + HAVING

Summarize your data by dividing it into groups and applying conditions.

✔ GROUP BY

→ Groups rows with the same values in specified column(s).



✔ HAVING

→ Filters groups based on a condition

Examples:

- SELECT Department, COUNT(*) FROM Employees GROUP BY Department;
- SELECT Department, AVG(Salary) FROM Employees GROUP BY Department HAVING AVG(Salary) > 60000;

Pro Tip:

✔ Combines well with aggregate functions like COUNT, SUM, AVG!

Pankaj Ikhar
Solution Architect & Mentor

SQL Zero to Hero – Day 5

ORDER BY + ALIAS

Let's organize our query results and make them easier to understand:

✔ ORDER BY

→ Sorts the results in ascending (ASC) or descending (DESC) order.

ID	Name	Salary
101	- - -	4K
105	- - -	7K
109	- - -	6K

✔ ALIAS

→ Renames columns or results.

Examples:

- SELECT Name, Salary FROM Employees
  ORDER BY Salary DESC;
- SELECT e.Name AS Employee, e.Salary AS Monthly_Pay
  FROM Employees e;
  WHERE e.Department = 'IT';
  ORDER BY Monthly_Pay DESC;

Pro Tip:

✔ Instead of sorting by column number, use an alias!

Pankaj Ikhar
Solution Architect & Mentor

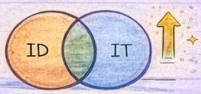
SQL Zero to Hero – Day 6

ORDER BY + ALIAS

Let's organize our query results and make them easier to understand:

✔ INNER JOIN

→ Returns matching rows from both tables



✔ LEFT JOIN

→ Returns all rows from the left table, and matching rows from the right table.

✔ RIGHT JOIN

→ Returns all rows from the right table, and matching rows from the left table.

✔ FULL JOIN

→ Returns all matching and non-matching rows from both tables.

Examples:

- SELECT Employees.Name, Departments.DeptName
  ORDER BY Salary DESC;
- SELECT e.Name AS Employee, e.Salary AS Monthly_Pay
  FROM Employees e;
  WHERE e.Department = 'IT';
  ORDER BY Monthly_Pay DESC;

Pro Tip:

✔ Instead of sorting by column number, use an alias!

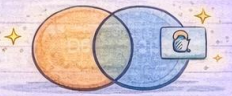
SQL Zero to Hero – Day 7

SQL Constraints

Rules that ensure your data stays valid and accurate.

✔ INNER JOIN

→ Ensures a column cannot have a NULL value.



✔ UNIQUE

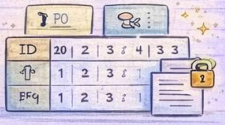
→ Ensures all values in a column are unique.

✔ PRIMARY KEY

→ Identifies each record uniquely.

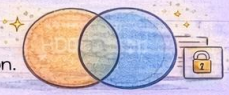
✔ FOREIGN KEY

→ Links a column to a primary key in another table.



✔ CHECK

→ Ensures a column meets a specific condition.



Examples:

- CREATE TABLE Employees
  ID INT PRIMARY KEY AUTO_INCREMENT,
  Email VARCHAR(50) UNIQUE NOT NULL);
- ALTER TABLE Orders ADD CONSTRAINT Orders_Check
  FROM Employees e;

Pro Tip:

✔ Use CONSTRAINT keyword to give names to rules for better readability!

SQL Zero to Hero – Day 8

CRUD Operations

Performing the four essential operations for managing data in SQL.

✔ CREATE

→ Add new records with INSERT.

✔ READ

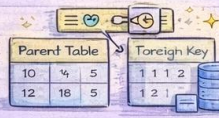
→ Retrieve records with SELECT.

✔ UPDATE

→ Modify records with UPDATE.

✔ FOREIGN KEY

→ Links a column to a primary key in another table.



✔ DELETE

→ Remove records with DELETE.

Examples:

- INSERT INTO Employees (Name, Salary) VALUES ("John Doe', 5000);
- SELECT * FROM Employees WHERE Salary > 4000;
- UPDATE Employees SET Salary = 5500 WHERE ID = 1;
  DELETE FROM Employees WHERE Salary < 3000;

Pro Tip:

✔ Always use WHERE keyword to give names to rules for better readability!

SQL Zero to Hero – Day 9

Performance Optimization

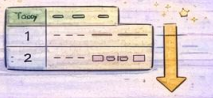
Practical tips for optimizing SQL query performance:

✔ **USE INDEXES** → SPEED UP SEARCHES

✔ **SELECT ONLY** WHAT YOU NEED
→ Avoid SELECT *

✔ **NORMALIZE** YOUR TABLES
→ Eliminate DUPLICATES

✔ **LIMIT** THE RESULTS
→ Reduce result size.



✔ **AVOID FUNCTIONS** IN WHERE CLAUSE
→ Keep it simple.

✔ **WRITE PROPER JOINS**
→ USE INNER JOIN WHEN POSSIBLE

✔ **OPTIMIZE SUBQUERIES**
→ Consider EXISTS OVER IN

Examples:

- INSERT INTO Employees (Name, Salary) VALUES ("John Doe", 5000);
- SELECT * FROM Employees e INNER JOIN Departments d ON e.DeptID = d.ID AND d.DeptName = "Sales";

Pro Tip:

✔ Always use WHERE to target specific records when updating or deleting!

SQL Zero to Hero – Day 10

Real-World SQL Tips

Practical tips for optimizing SQL query performance:

✔ **USE INDEXES → SPEED UP SEARCHES**

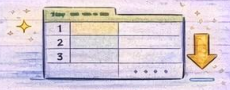
→ Know the data needs of your team/organization.

✔ **PRACTICE SAFE UPDATES & DELETES**

→ Use WHERE & test on sample data first!

✔ **AUTOMATE TASKS**

→ Schedule regular data backups and maintenance.



✔ **DOCUMENT YOUR QUERIES**

→ Write clear, concise comments for others to understand.

✔ **STAY UPDATED**

→ Keep learning and improve your SQL skills.

✔ **COLLABORATE EFFECTIVELY**

→ Communicate clearly with team members.

✔ **USE STORED PROCEDURES**

→ Organize and reuse common query logic.

✔ **SENSITIVE DATA PROTECTION**

→ Use encryption, masking, and user permissions.

Keep debugging fun and be patient—great SQL queries take time and practice! 😊

Pankaj Ikhar
Solution Architect & Mentor

SQL Zero to Hero - Day 11

Common Table Expressions (CTE)

A way to create temporary result sets for use within a SELECT, INSERT, UPDATE, or DELETE statement.

- ✔ Improve readability and maintainability.
- → Break down complex queries into simple chunks.
- ✔ Easily reference the result within the same query.
- ✔ Avoid using subqueries
- → for better performance.

Example:

```
--- Define the CTE
WITH HighEarners AS (
    SELECT Name, Salary
    FROM Employees
    WHERE Salary > 6000
)
--- Use the CTE in another query
SELECT * FROM HighEarners
    ORDER BY Salary DESC;
```

Pro Tip:

✔ CTEs can be used multiple times in the same SQL statement!

Pankaj Ikhar
Solution Architect & Mentor

SQL Zero to Hero - Day 12

Window Functions

Window functions perform calculations across a set of table rows related to the current row, providing advanced analytics capabilities.

- ✔ **ROW_NUMBER()**
  → Assigns a unique sequential number.

- ✔ **RANK()**
  → Assigns a rank with gaps for equal values.

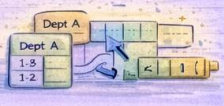
ID	Z	8	3	
1	1	2	2	1
2	1	2	3	2

- ✔ **DENSE_RANK()**
  → Ranks without gaps.

ID	Z	8	2	
1	1	2	2	1
2	1	2	3	2

- ✔ **LAG()/LEAD()**
  → Access data from previous or next rows.

- ✔ **PARTITION BY - OVER()**
  → Partitions data into groups for analysis.



Example:

```
SELECT Name, Salary, DeptID,
       ROW_NUMBER() OVER (PARTITION BY DeptID
          ORDER BY Salary DESC) AS RowNum
FROM Employees;
```

Name	Salary	DeptID	RowNum
Alice	7000	1	1
Bob	6000	1	2
Carol	8000	2	1

Pro Tip:

✔ Window functions don't collapse rows; they let you view data with calculated columns!

SQL Zero to Hero – Day 13

Subqueries

A subquery is a query nested inside another SQL query.

✔ IN Subqueries

→ Checks if a value exists within a list of results.

✔ EXISTS Subqueries

→ Tests for the existence of rows in the result set.

✔ Correlated Subqueries

→ Refers to columns in the outer query.

Example:

```
SELECT Name, Salary
    FROM Employees
      WHERE Salary (Salary)
      FROM  Employees
    ),
```

Name	Salary
Alice	7000
Carol	8000

Pro Tip:

✔ Keep it efficient! Use subqueries to replace joins for specific problems.

Pankaj Ikhar
Solution Architect & Mentor

SQL Zero to Hero – Day 14

Views + Stored Procedures

Learn how Views and Stored Procedures can make your SQL work easier and more efficient:

✔ Views

→ Virtual tables based on queries.

→ Useful for simplifying complex queries.

```
CREATE VIEW EmployeeDetails AS
    SELECT E.Name, E.Salary, D.DeptName
    FROM Employees E
    JOIN Departments D ON E.DeptID = D.DeptID;
),
```

Name	Salary	Dept
Alice	7000	Sales

✔ Stored Procedures

→ Reusable, stored, named SQL code that performs actions.

```
CREATE PROCEDURE AddEmployee(
    IN EmpName VARCHAR (100),
    IN EmpSalary.DECIMAL (10, 2),
    IN EmpDeptID INT
BEGIN
    INSERT INTO Employees (Name, Salary, DeptID)
      VALUES (EmpName, EmpSalary, EmpDeptID);
END;
CALL AddEmployee('John', 5000, 1);
```

Pro Tip:

✔ Views ore handy for read-only tosks, while Stored Procedures can modify data.!

Let's Connect!

For interview preparation, short & long term mentorship, and design & architectural consulting.

- Job interview preparation
- Short & long term mentorship
- Design & architectural consulting

Feel free to reach out:

greenleaf.educations@gmail.com

/in/pankaj-ikhar

@Pankajikkar

Pankaj Ikkar

Solution Architect & Mentor

Repost | Share | Save | Comment

Pankaj Ikkar
Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting